六篇工业表面缺陷检测论文阅读报告

学习主题：小缺陷特征、模型压缩、知识蒸馏与设备部署 整理日期：2026 年 10 月 19 日 阅读材料：用户提供的六篇本地 PDF；Markdown 辅助定位，关键结果回查 PDF。 定位：面向正在学习计算机视觉与工业缺陷检测的读者，既解释“作者做了什么”，也解释“每一步有什么用”。

阅读说明与学习目标

本报告不是六篇摘要的拼接。它围绕一条问题链展开：缺陷太小、背景太复杂，所以需要更好的特征；设备算力有限，所以需要压缩；压缩影响精度，所以需要恢复；最后必须在真实设备和完整流程上检验。

报告使用三种明确口径：**原文结果**指作者报告的数据，不代表本报告复现实验；**原理解释**用于帮助理解，不等于已经证明该机制导致了实验收益；**阅读判断**是根据实验设计、表格和叙述作出的评估。报告未开展训练复现，也未检索出版后的勘误、代码更新或独立复现。

读完后，你应当能够：

1. 画出检测器 Backbone、Neck、Head 的信息流，并解释小目标为什么容易丢失。
2. 区分轻量结构、剪枝、蒸馏、量化分别改变了什么。
3. 独立读懂参数量、FLOPs、模型文件、延迟和 mAP 的关系。
4. 从消融实验判断一个模块的收益、代价和证据边界。
5. 提出一个可以验证的小实验，而不是简单拼接论文模块。

建议分三轮阅读：第一轮读第 1、2 节建立知识地图；第二轮按 Chen → Cao → Lu → PCB Wang → Shi → 边缘 Wang 的顺序学习；第三轮完成第 10、11 节的练习，再回原文核对。

1. 六篇论文在解决什么问题

1.1 论文身份与阅读编号

编号	论文与出处	报告简称
P1	Zhenling Wang 等，A Lightweight and High-Precision PCB Surface Defect Detection Method Based on YOLOv8，Journal of Imaging，2026，12:266；DOI：10.3390/jimaging12060266	PCB Wang
P2	Yiqing Cao 等，Lightweight metal surface defect detection algorithm based on pruning and knowledge distillation，Scientific Reports，2026，16:25700；DOI：10.1038/s41598-026-57496-0	Cao
P3	Qipeng Chen 等，Research on the Construction of an Efficient and Lightweight Online Detection Method for Tiny Surface Defects through Model Compression and Knowledge Distillation，Electronics，2024，13:253；DOI：10.3390/electronics13020253	Chen
P4	Kun Lu 等，RDDPA: Real-time Defect Detection via Pruning Algorithm on Steel Surface，ISIJ International，2024，64(6):1019－1028；DOI：10.2355/isijinternational.ISIJINT-2023-360	Lu / RDDPA
P5	Fengdong Shi 等，MBDANet: a multi-scale balanced dynamic alignment detection network based on knowledge distillation for small objects，Scientific Reports，2026，16:21053；DOI：10.1038/s41598-026-51781-8	Shi / MBDANet
P6	Zishuo Wang 等，Workpiece surface defect detection based on YOLOv11 and edge computing，PLOS One，2025，20(7):e0327546；DOI：10.1371/journal.pone.0327546	边缘 Wang

下文页码统一指本地 PDF 的第几页。P4 的 PDF 第 1 页对应期刊印刷页 1019，避免两种页码混淆。编号表示阅读来源，不表示质量排名。

1.2 一张问题地图

工业表面图像
- → 缺陷像素少、对比度低、背景复杂
- → 特征保留与融合：P1、P5
- → 模型规模或计算开销过大
- → 结构化剪枝：P2、P3、P4
- → 压缩后的性能恢复：P2、P3 用蒸馏，P4 调训练策略
- → 部署精度与时延：P3 的 AGX Xavier、P6 的 RK3568

这些方法并不构成同一个现成算法。把它们拼接到一起，仍然需要重新训练、排查模块冲突和验证实际收益。

1.3 各篇最值得带走的思想

论文	核心问题	最值得学习的一件事
PCB Wang	PCB 小缺陷难检，同时希望比大基线更省资源	先用轻量骨干节约开销，再将部分预算用于高分辨率检测
Cao	增强版检测器精度较高，但不够紧凑	剪枝负责减规模，特征与输出蒸馏负责恢复性能
Chen	航空零件微小缺陷的在线检测与嵌入式运行	稀疏训练、通道剪枝、层剪枝、蒸馏的完整流程
Lu	传统剪枝训练流程复杂，小 GPU 上速度不足	质疑“必须先在目标任务稀疏训练”，并研究剪后训练预算
Shi	小目标特征丢失、多尺度失衡和分类定位冲突	按信息流逐段改进，同时检验模块组合是否真的有效
边缘 Wang	数据少、依赖云端、边缘设备部署	训练与推理分工、INT8 转换，以及如何审查部署证据

2. 读这六篇前，先弄懂八个基础概念

2.1 检测、分类、分割与异常检测

分类回答“这张图有没有缺陷、是哪种缺陷”；目标检测还给出矩形框位置；分割给出像素级轮廓。异常检测通常利用正常样本或有限异常监督，寻找偏离正常模式的区域。具体监督方式因方法而异。

六篇主要围绕带类别和框标注的缺陷目标检测。它们不能直接证明同一种方法在无监督异常检测、开放类别识别或像素分割上也有效。比如检测框覆盖一条裂纹，并不意味着已经测出了裂纹长度和真实面积。

2.2 Backbone、Neck、Head 分别有什么用

```
图像 [B, 3, H, W]
  → Backbone：提取边缘、纹理与更抽象的语义
  → 多尺度特征 [B, C_i, H_i, W_i]
  → Neck：融合不同分辨率和语义层级
  → Head：预测类别与位置
  → 解码、阈值过滤、必要的 NMS
  → 最终缺陷框
```

B 是批大小，C 是通道数，H、W 是空间尺寸。通道可以理解为不同的特征响应，不应简单当成某个固定缺陷类别。

浅层通常保留较细的空间信息；深层通常包含更强的语义与更大的感受野。小目标检测需要两者结合：只有细节，可能把正常纹理当成缺陷；只有语义，微小缺陷可能已经被下采样削弱。

2.3 为什么小缺陷容易丢失

以 640×640 图像中的一个 8×8 像素缺陷为例，步长 8 的特征图是 80×80，该缺陷约对应一个空间单元；步长 4 的特征图是 160×160，约对应 2×2 个空间单元。

这是帮助理解的近似映射。真实特征受卷积、感受野和插值共同影响，不是简单缩小图片；它说明的是更粗的采样会增加小缺陷信息损失风险。

P2 分支保留更高分辨率，但 160×160 的位置数是 80×80 的四倍。它可能提高召回，也可能增加内存访问、背景误报和计算。需要用小目标分组指标、漏检例子和实际延迟验证，而不只看总 mAP。

2.4 Precision、Recall、AP、mAP 怎么读

```
Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
IoU       = 预测框与真值框的交集面积 / 并集面积
AP        = 对一类目标的 Precision-Recall 曲线计算面积或离散近似
mAP       = 各类别 AP 的平均值
```

TP 的判定还需要正确类别、IoU 阈值和一对一匹配等规则。同一真值被重复检测时，多余框可能成为 FP。

假设有 100 个真缺陷，检出 90 个，同时产生 10 个误报：Precision=90%，Recall=90%。如果降低置信度阈值，可能多找回几个缺陷，也多产生一些误报；扫描不同阈值形成 PR 曲线。

AP 不是某一个阈值下 Precision 和 Recall 的平均数，也不是分类准确率。 mAP@0.5 在 IoU=0.5 的匹配条件下计算；mAP@0.5:0.95 再对多个 IoU 阈值求平均，定位要求更严格。还要区分评估 IoU 与 NMS IoU，两者用途不同。

从 70.0% 提升到 72.0%，应写“提高 2.0 个百分点”；相对提升约为 2.86%。报告尽量使用百分点，避免混淆。

2.5 参数量、FLOPs、模型文件和显存是四件事

指标	回答的问题	不能直接推出什么
参数量	有多少可学习权重	不能直接推出实际延迟
FLOPs	一次前向传播的理论浮点运算规模	不能反映全部访存和算子调度代价
模型文件大小	序列化到磁盘的文件多大	不等于参数个数，也不等于运行显存
运行内存/显存	推理或训练期间占多少空间	不只由权重决定，还包括激活、缓存等
延迟与吞吐	一次处理等多久、单位时间处理多少	需要明确 batch、计时范围和硬件

同样数量的权重，以 FP32、FP16、INT8 存储，裸权重字节数不同；模型文件还可能保存图结构、元数据或其他状态。因此不能把“4.19 MB”写成“4.19 M 参数”。

普通卷积在忽略偏置时，参数量为 k×k×C_in×C_out；计算规模还与输出 H×W 相关。某些工具将一次乘加算作一次，有些按两次计数。跨论文比较 FLOPs 前也要检查这一点。

2.6 剪枝为什么可以减规模

非结构化剪枝把分散的权重设为零；若运行库不支持稀疏计算，张量形状可能不变，速度不一定变快。结构化剪枝删除整组通道、卷积核或层，直接改变张量尺寸，更容易交给常规硬件执行。

例如一个 3×3 卷积从 64 输入通道、128 输出通道变成 64 输入、64 输出，参数由 73728 变为 36864；下一层原本接收 128 通道，也必须同步调整。若两条分支需要相加，尺寸必须保持兼容。这就是论文里处理 shortcut 和通道依赖的实际意义。

2.7 蒸馏到底给学生增加了什么

```
训练阶段：同一图像 → 教师与学生 → 真值监督 + 教师指导
部署阶段：图像 → 学生 → 预测结果
```

教师提供的信息可以是类别概率、预测框、中间特征或它们之间的关系。蒸馏一般改变学生的训练约束，教师通常不参与学生部署推理。因此“蒸馏不增加推理复杂度”有条件地成立，但训练成本通常增加。

一个便于理解的总目标为：

```
L_total = L_detection + a × L_feature + b × L_cls_KD + c × L_box_KD
```

这是统一解释形式，不是六篇共享的原始公式。权重太大可能让学生追随教师错误；背景位置太多可能淹没小缺陷信号；特征通道不同还需要选择兼容层或建立映射。蒸馏也不保证一定提升性能，P5 的实验提供了负收益例子。

2.8 量化为什么不等于剪枝

剪枝主要减少网络结构或连接；量化主要减少数值表示的精度。用于解释的通用表达式是：

```
q = clip(round(x / s) + z, q_min, q_max)
x_hat = s × (q - z)
```

s 是尺度，z 是零点。需要用代表性数据确定数值范围。INT8 的有符号范围通常是 -128～127，无符号 8 位通常是 0～255；两者都包含 256 个可表示值。实际格式和每通道/每张量量化方式取决于工具和模型。

量化误差可能影响低对比度缺陷和边界回归。能成功导出模型，不代表精度和速度已通过验证。

3. P1：PCB Wang——高分辨率特征与轻量骨干的预算分配

3.1 研究问题与方法总览

研究对象是 PCB 表面六类缺陷，包括缺孔、鼠咬、开路、短路、毛刺和杂铜。作者以 YOLOv8L 为基线，希望在减少其复杂度的同时提高小缺陷检测能力。[P1，PDF 第 1–3、8 页]

YOLOv8L
→ RepGhostBottleneck：降低部分特征提取开销
→ Coordinate Attention：结合位置与通道响应
→ WIoU：调整边界框回归监督
→ P2 检测分支：利用更细的空间特征

3.2 四个改动分别有什么用

RepGhostBottleneck。 原文用较廉价的特征生成、深度卷积及特征组合替换 C2f 中的 Bottleneck。学习重点是减少高成本的重复特征计算，并利用残差连接传递信息。不要仅凭模块名就认定其导出模型一定完成了某种重参数化融合；需要进一步看实现与部署图。[P1，第 3–4 页]

CA 坐标注意力。 分别沿横向、纵向汇聚信息，保留某一方向的位置线索，再生成权重调节特征。对细小缺口或铜线边缘，知道响应位于哪里很有价值。注意力并不直接保证关注正确，也可能增强背景，需要通过消融和错误案例检验。[P1，第 4–6 页]

P2 检测头。 在 640×640 输入下增加步长 4、160×160 特征分支，使极小目标有更多空间采样位置。它直接对应“浅层细节保留不足”的问题，同时带来额外计算代价。[P1，第 7–8 页]

WIoU。 原文列出以 IoU 损失结合中心距离权重的表达式。学习时关注回归误差如何被重新加权，以及部分几何量为何从梯度图中分离。不要把所有 WIoU 版本、动态聚焦机制统统归到本篇的实际实现中；本文列出的公式和代码实现仍需一一核对。[P1，第 6–7 页]

3.3 数据与实验条件

原文使用公开 PCB 图像 693 张、工业采集图像 6319 张，共 7012 张。按 8:2 划分训练和验证，并声明同一 PCB 样本不同时进入两组。工业数据在固定工作距离、曝光与照明条件下采集；这有助于控制成像波动，但不能替代跨产线验证。[P1，第 8–9 页]

主要配置：输入 640×640，batch=8，200 epochs，SGD，RTX 4070。工业部分不能直接公开，独立复现受到数据获取限制。文中未在该数据划分说明中给出单独的最终测试集。[P1，第 9 页]

3.4 关键结果应该怎样读

对照点	mAP@0.5	参数量	FPS	来源
YOLOv8L 基线	91.0%	43.7 M	30	表 4
替换 RepGhost 后	93.5%	27.3 M	36	表 4
再加 CA 后	94.7%	27.6 M	36	表 4
表 4 最终配置	97.6%	34.0 M	34	表 4，配置标记有疑点

最终 mAP@0.5:0.95 为 91.0%。相对大基线，参数下降约 22.2%，mAP@0.5 增加 6.6 个百分点；最终模型依然有 34 M 参数。作者所说“轻量”是相对于 YOLOv8L，而不是说明它是极小网络。[P1，第 13 页表 4]

TensorRT 实验另有一组口径：FP32 为 19.6 ms、51 FPS、97.6% mAP@0.5；FP16 为 15.4 ms、65 FPS、97.4%；INT8 为 12.2 ms、82 FPS、95.8%。这是精度模式与推理后端改变后的结果，不宜与表 4 的 34 FPS 直接合并成一个条件。[P1，第 14 页表 5]

3.5 值得肯定与需要核查的地方

有价值的地方是预算分配：轻量骨干先释放资源，再让高分辨率分支获得更强的小目标表达。最终参数从轻量阶段的约 27 M 回升至 34 M，本身就说明小目标精度需要付出代价。

但本地 PDF 存在内部不一致：

1. 表 4 最后一行的 CA 列没有勾选，而正文把最终模型描述为结合 CA 等改进。最终精确配置应回到代码或作者说明核实，不能把表格最后一步直接当成严格的“只加入 P2”。
2. 表 3 的 YOLOv8 六类 AP 平均约为 94.7%，而表 4 基线为 91.0%；若属于相同模型与评估划分，需要解释差异。
3. 正文称平均速度约增加 2 FPS，但表 4 的 30→34 对应增加 4 FPS。
4. 第 15 页把极端长宽比问题归因于预设 anchor box，不能作为学习 YOLOv8 的通用准确表述；阅读时应区分 anchor-free 检测与仍存在的特征点、框回归表示。

这些问题不自动证明方法无效，但使“每个模块贡献多少”和“可精确复现的最终配置”需要更谨慎地表述。

学完应能回答： 为什么节省下来的参数还要花回一部分？为什么增加 P2 后精度可能升高而速度下降？为什么 97.6% 不能直接与其他论文的 mAP 排名？

4. P2：Cao——剪枝与多层次蒸馏的配合

4.1 研究问题：先提高精度，再压缩是否可行

作者已有增强版 SAF-YOLOv8n，在 GC10-DET 上比原始 YOLOv8n 更准确，但参数量和 FLOPs 更高。本文的任务是压缩这个增强模型，并尽可能保留其检测能力。[P2，第 2 页]

SAF-YOLOv8n 的 SI-C2f、FASFFHead 和 ADown 属于本文采用的前期模型基础。阅读贡献时应把“基础检测器的改进”和“本文新增的压缩与蒸馏方案”分开。

```
完整 SAF-YOLOv8n
  → 按 L1 范数评估卷积核重要性
  → 结构化全局剪枝
  → 压缩模型作为学生，完整模型作为教师
  → 特征 Mimic + 分类蒸馏 + CIoU 定位蒸馏
  → 紧凑学生模型
```

4.2 L1 范数剪枝：究竟删什么

对一个输出通道对应的卷积核，累计其全部权重绝对值，作为重要性近似：

```
score_j = sum(abs(W_j))
```

较小分数被认为贡献可能较低。全局剪枝将多个候选层的分数放在一起排序；局部剪枝则各层分别决定比例。全局方法较灵活，但不同层的权重尺度会影响可比性。

本文并非随意删除所有层。终端检测输出、DFL、部分融合与重建相关层受到保护，以避免破坏输出含义和网络约束。[P2，第 3–4、7–8 页]

学习要点： L1 范数是重要性启发式，不是“此通道必定没有作用”的证明。剪枝需要与结构依赖处理和训练恢复配合。

4.3 BFCD-KD：从三个层次指导学生

中间特征 Mimic。 对选定的对应特征计算归一化均方误差，使学生接近教师的特征表达。它不只是要求最终类别一致，还要求内部表征具有相似性。原文说明选择空间与通道尺寸相同的层；对剪后通道变化的其他层，不能默认逐元素比较就成立，应检查实际对齐选择与适配实现。[P2，第 5 页、图 4]

分类蒸馏。 对教师和学生的 Sigmoid 概率计算差异，并以差异大小作为权重，再计算二元交叉熵。学生已接近教师的位置权重较低，差异较大的位置权重较高。大差异也可能来自教师错误，因此仍需要真值监督。[P2，第 6 页]

定位蒸馏。 在解码后的框坐标上，用 CIoU 衡量师生位置、尺度和形状的差异。与只学习类别相比，这直接约束“框得准不准”。[P2，第 6 页]

这三个层次对应“看到了什么特征”“认为是什么类别”“认为在哪里”。部署时使用学生，教师不参与推理；训练却需要额外计算与显存。

4.4 关键实验：必须分清两个基线

GC10-DET 覆盖十类金属表面缺陷。压缩训练使用 640×640、batch=16、300 epochs，实验 GPU 为 RTX 4090。论文将剪枝率定义为原模型被移除 FLOPs 的比例，并不等同于移除通道或参数的比例。[P2，第 7 页]

模型	mAP@0.5	mAP@0.5:0.95	参数量	FLOPs
YOLOv8n	67.2%	33.0%	3.0 M	8.1 G
完整 SAF-YOLOv8n	70.8%	35.4%	4.1 M	14.7 G
约 50% FLOPs 剪枝后	67.5%	32.4%	2.4 M	7.2 G
剪后 + 特征蒸馏	70.1%	33.7%	2.4 M	7.2 G
剪后 + 检测头蒸馏	68.4%	32.9%	2.4 M	7.2 G
剪后 + BFCD-KD	70.6%	34.8%	2.4 M	7.2 G

来源：P2，第 8 页表 2、第 10 页表 3。

相对完整 SAF 模型，最终参数减少约 41.5%，FLOPs 减少约 51.0%，mAP@0.5 仅下降 0.2 个百分点；相对原始 YOLOv8n，参数减少 20%，FLOPs 减少约 11.1%，mAP@0.5 增加 3.4 个百分点。

蒸馏使剪后 mAP@0.5 从 67.5% 回到 70.6%，恢复 3.1 个百分点；它没有让这个固定学生从 2.4 M 参数变得更小。真正减结构的是剪枝。

4.5 阅读判断

本篇适合学习完整的“增强模型→压缩→性能恢复”证据链。特征蒸馏单独已恢复较多性能，组合再增加 0.5 个百分点；是否值得增加训练成本，要看重复实验和具体应用。

需要注意三点：第一，第 7 页部分叙述把参数与 FLOPs 的数值顺序写反，应按表格列标题读取。第二，工业相机、照明和 PyQt5 界面证明做了系统集成，但示例置信度 0.92 不是完整测试集准确率。第三，本文结尾仍把边缘设备部署优化列为后续工作，不能把 RTX 4090 实验和界面展示理解为已量化证明某块低功耗边缘板满足实时要求。[P2，第 11 – 12 页]

学完应能回答： 为什么相对 SAF 的“减少 51%”与相对 YOLOv8n 的“减少 11.1%”可以同时成立？为什么不能将 BFCD-KD 的收益全部归给特征蒸馏？

5. P3：Chen——从稀疏训练走到嵌入式在线检测

5.1 研究对象与整条技术路线

研究对象是航空产品偏心锥形工件表面的微小加工缺陷。共采集 1200 张图像，凹痕、裂纹、毛刺、断裂四类，各 300 张；输入 416×416。检测基础为 YOLOv4，并结合数据增强和 K-means++ 先验框聚类。[P3，第 3－8、15－16 页]

采集并标注图像
→ 训练 YOLOv4 检测模型
→ 对 BN 的 gamma 加 L1 稀疏约束
→ 按 gamma 剪通道
→ 按层的 gamma 统计量剪部分结构
→ 蒸馏与微调恢复性能
→ 在 Jetson AGX Xavier 上测试

5.2 数据增强和 anchor 聚类有什么用

Mosaic 将多张训练图缩放、裁剪并组合，增加尺度和背景组合变化；它不会把 1200 张源图变成同等数量的新独立工件。K-means++ 根据缺陷框尺寸建立更适合数据的先验框，减少初始框与真实缺陷尺度的失配。这一思路针对本篇的 anchor-based 检测流程，不应机械搬到所有 YOLO 版本。

文中方法章节使用 Mosaic，但结果章节又出现 CutMix 表述，复现时需要进一步确定实际增强配置。[P3，第 4、16 页]

5.3 为什么 BN 的 gamma 能用于剪枝

BN 的简化表达为：

```
y_c = gamma_c × normalized(x_c) + beta_c
L_sparse = L_detection + lambda × sum(abs(gamma_c))
```

当某通道 gamma 很小时，归一化输入变化对输出的影响通常较弱。L1 正则鼓励更多 gamma 接近零，使通道更容易排序和选择。

但小 gamma 只是代理指标；beta、后续层及分支交互仍可能影响作用。不能把“gamma 小”理解为数学上证明删掉绝对无害。因此需要完整前向检查和剪后评估。[P3，第 10 页]

通道剪枝主要缩窄网络；层剪枝进一步减少深度或残差结构。作者避开部分直接连接 shortcut 的层以保持维度一致，并依据 gamma 均值选择部分层，报告剪去层 59、90。删除残差结构时不是任意抽掉一层，而要同时处理相关连接。[P3，第 11、21 页]

5.4 蒸馏：软标签与定位约束

分类部分使用带温度的 Softmax 分布和真值监督。温度 T>1 使概率更平滑，学生能看到类别间的相对相似性，而不只看到 one-hot 标签。文中的 z 在理解时应视作 Softmax 前的分数/logits，不能再当作已经归一化的类别概率。

定位部分使用有界回归思想：利用教师相对真值的误差衡量学生是否仍需被额外约束，避免学生已经足够好时继续过度约束。这与无条件令所有师生坐标相等不同。[P3，第 12 – 13 页]

学习重点： 蒸馏指导应针对检测任务设计；分类概率与框回归的量纲、目标、匹配方式并不相同。照搬一个分类 KL 损失不能自动覆盖检测蒸馏的所有问题。

5.5 结果：把每个阶段分开

阶段	参数个数	模型大小	mAP / 原表精度列	说明
原始检测模型	63,953,841	244	97.50%	表 5
通道剪枝后	1,107,904	4.30	95.14%	表 5
再做层剪枝	1,080,754	4.19	95.14%	表 5
蒸馏微调后的模型	采用压缩结构	4.19 MB	97.41%	正文及表 6

表 5 的大小表头写为 “M”，表 3、表 6 以 MB 表示模型大小；结合全文使用，244→4.19 应按模型文件大小理解，不能写成 244 M→4.19 M 参数。来源：P3，第 18、22 – 23 页。

原模型在单张 RTX 2080 Ti 上报告 32.8 FPS；压缩后表 6 报告 68.0 FPS，约为 2.07 倍。在 Jetson AGX Xavier 上报告 47.8 FPS；这是一条实际嵌入式设备证据，而不是从桌面 GPU 外推。[P3，第 18、23 页]

文件大小减少约 98.3%，参数由约 64 M 降到约 1.08 M；FPS 没有提高约 58 倍。原因是参数节省比例不等于总执行时间节省比例，运行还受激活、访存和非卷积开销等影响。

5.6 阅读判断：优点与评估问题

优点是从数据采集到设备测试的链条比较完整，能用来学习工程流程；不足是数据源较单一，通用性仍需更多产品和成像条件验证。

较重要的评估问题： 第 16 页明确写测试集用于超参数调整和性能评估，第 17 页又按固定间隔在测试集上计算 mAP。若没有额外未参与选择的最终测试集，就存在测试集参与模型选择的问题，97.41% 不能视作严格盲测结果。

此外，表 5 的未压缩模型为 97.50%，而微调段落用“96.05% 比未压缩模型低 1.29%”描述；两个数字并不完全对应。学习时保留各自来源，不拼出没有依据的统一曲线。

学完应能回答： 为什么稀疏训练不等于已经完成加速？为什么蒸馏把 95.14% 恢复到 97.41% 后，模型大小可以不变？为什么测试集不能一边调参一边作为最终成绩？

6. P4：Lu——简化剪枝流程，并重新设计训练预算

6.1 研究问题与检测器结构

RDDPA 关注普通 GPU 上钢表面缺陷检测的速度与精度，并试图简化常见的“目标任务训练、稀疏训练、剪枝、微调”流程。[P4，第 1–3 页]

检测器采用 Tiny-Darknet 骨干、带上采样与变形卷积的特征处理和中心点检测。Head 预测类别中心热力图、中心偏移与宽高。热力图回答“哪里像某类缺陷的中心”；偏移补偿网格离散误差；宽高恢复框尺寸。推理包含局部峰值选择与 NMS。[P4，第 3–4 页]

6.2 这篇的核心不是新增一个蒸馏损失

作者观察到，在本文实验中，从 ImageNet 预训练权重获得的 BN 通道重要性，可以用来直接确定紧凑骨干结构，不必先在 NEU-DET 上经历额外稀疏训练。[P4，第 4–5、8 页]

```
ImageNet 预训练 Tiny-Darknet
  → 根据 BN 缩放因子排序并剪枝
  → 通道数调整为 16 的倍数
  → 根据剪后骨干适配 Neck
  → 用调整后的策略训练检测器
```

这里的“from scratch”“end-to-end pruning”是作者在其流程语境中的用词，不应理解为从未使用任何预训练信息：剪枝依据明确来自 ImageNet 预训练权重，也不应默认是把离散通道删除直接放进一个完全可微的端到端优化器。

通道数对齐硬件友好的粒度，即便少剪掉一点，也可能更利于实际并行执行。**最少参数的网络未必是最快网络。**

6.3 LS：为什么剪完后要改变训练

作者发现高压缩模型沿用原来的训练配置时可能训练不足，于是根据剪枝程度调整初始学习率、衰减节点和训练轮数。其经验策略在本篇中有效，但不是“模型越小就应该无限增大学习率”的通用定律。[P4，第 5、8 页]

对小模型使用更多 epochs，单步仍更便宜。判断训练效率应同时看总计算量或墙钟时间，而不能只比较 epochs 个数；训练稳定性也需要独立验证。

6.4 数据与关键结果

NEU-DET 六类，每类 300 张，总计 1800 张，70% 训练、30% 评估。输入 384×384，训练 batch=32，在 Tesla V100 上训练，Titan X 与 GTX 960M 上测试。原文还描述推理中的尺度和翻转设置，复现速度时须进一步明确它们与计时的关系。[P4，第 5 页]

模型	参数量	FLOPs	GTX 960M FPS	mAP
RDDPA 1.0×	6.7 M	6.2 G	21.6	79.9%
RDDPA 3.0×	2.2 M	4.4 G	27.0	79.9%
RDDPA 4.3×	1.6 M	3.5 G	30.2	79.7%
RDDPA 10.8×	0.6 M	1.8 G	40.1	79.2%

来源：P4，第 6 页表 2。10.8× 模型在 Titan X 上为 103.7 FPS；40.1 与 103.7 是两种 GPU 的结果。[P4，第 7 页表 4]

“10.8×”是作者标注的权重压缩等级，不是速度提升 10.8 倍。GTX 960M 的速度比为 40.1/21.6≈1.86。

训练策略消融很关键：10.8× 模型不用 LS 为 76.8% mAP，使用 LS 后为 79.2%，恢复 2.4 个百分点。3.0× 对应 79.0%→79.9%，4.3× 对应 78.1%→79.7%。这表明最终结果的部分收益来自训练设置，而非只来自剪枝规则。[P4，第 9 页表 5]

6.5 阅读判断

本篇最有价值的研究习惯，是把“传统流程中的某一步是否真的必要”当成可验证问题。它不依靠堆叠模块，而是研究剪枝结构、预训练和训练预算之间的关系。

边界也清楚：主要验证集中于一个缺陷基准；没有在这份材料中提供 RK3568 结果；更少 epochs 或取消一个阶段不自动等于更低总能耗。若要移植到自己的模型，应该比较相同总训练预算下的剪后结构、随机初始化结构与蒸馏结构。

学完应能回答： 为什么“用了预训练骨干”与“没有先训练完整目标检测器”并不矛盾？为什么评估剪枝方法时需要控制训练预算？

7. P5：Shi——沿着特征流改进小目标检测

7.1 总体思路：问题分解到三个位置

MBDANet 以 YOLOv8n 为基础，将问题分为下采样丢细节、跨尺度融合失衡、分类定位需求不一致，再用 BCKD 提升学生性能。[P5，第 2－4 页]

输入
→ RFD：浅层和深层多路径下采样
→ MFBPN：全局上下文与跨尺度平衡
→ TDAH：共享特征后按分类/回归需要动态处理
→ 输出

训练时：YOLOv8s 教师 → BCKD → MBDANet 学生

7.2 RFD：下采样不只保留一种信息

RFD 包含浅层 SRFD 和深层 DRFD，综合深度可分离卷积、切片下采样和最大池化等路径。切片路径帮助重排和保留局部采样信息，池化与卷积路径提供互补响应，再融合。[P5，第 4－5 页]

学习时不要只背缩写，而要问：每个分支输出的 H、W、C 是什么？拼接后通道如何变化？随后有没有通道压缩？这些决定了信息流和真实成本。

RFD 的单模块实验将 mAP 从 93.9% 提升到 96.2%，但 GFLOPs 从 8.1 增至 9.6。它增强特征的同时确实增加了计算，而非每个子模块都更轻。[P5，第 16 页表 6]

7.3 MFBPN：融合不是简单拼接

MFBPN 由 PCE、RCM、FBM、DIF 协作完成多尺度上下文处理与融合：[P5，第 6－7 页]

子模块	主要作用	带着什么问题去看公式
PCE	汇聚不同尺度上下文，再分发到分支	不同尺寸如何对齐，拼接维度是什么
RCM	通过轴向/矩形自校准注意力调整特征	增强了哪些空间关系，保留了什么残差
FBM	用高层语义权重调节低层细节	相乘的两个张量如何对齐
DIF	插值与通道映射后跨层融合	上采样和 1×1 卷积各改变什么

你需要理解的是“粗尺度的语义帮助筛选细尺度的细节”；不要把插值理解为能凭空恢复原来已经消失的真实缺陷像素。

7.4 TDAH：分类与定位既共享又分工

分类偏向判断缺陷语义，定位需要精确边界与几何位置。完全独立的两个分支可能冗余，完全共享又可能造成需求冲突。TDAH 先共享部分 GN 卷积特征，再通过任务分解、层注意力、动态可变形卷积和分类注意力分别调节。[P5，第 7－8 页]

GN 在单个样本的通道分组内计算统计量，对 batch 规模的依赖较小，适合本文 batch=4 的设置。它与“分组卷积”不是一回事，看到文中的 shared group convolution 表述时要辨清实际操作。

动态算子、GN、小尺寸算子与访存行为都可能影响执行效率。算子是否被目标设备支持，是后续部署必须单独解决的问题。

7.5 BCKD：不把它当作必然有效的魔法

本篇使用 YOLOv8s 作为教师。分类部分按师生概率差异加权二元交叉熵；定位部分基于师生预测框 IoU 构造监督。与 Cao 相比，两者都有输出指导，但 Cao 还加入特征 Mimic，定位项使用 CIoU；不能将两种方案视为完全相同。[P5，第 9 页]

原文把 BCKD 作为已有知识蒸馏方法引入。不要把它列成作者从零提出的全新通用蒸馏框架。检测点与框如何对应、背景是否参与、权重如何设置，仍是复现要点。

7.6 数据与实验：优先按表格对应

本文设置为 640×640、100 epochs、batch=4，GPU 在原文中写为 RTX 2080。PCB 源图 693 张，增强后 2818 张，再按 8:1:1 随机划分。另在钢表面缺陷和 DeepPCB 上实验。[P5，第 10－11、13－14 页]

数据集	YOLOv8n	MBDANet 无蒸馏	MBDANet+BCKD	相对基线增益
PCB	93.9%	95.6%	96.6%	2.7 个百分点
钢表面缺陷	73.1%	74.9%	76.8%	3.7 个百分点
DeepPCB	94.0%	98.2%	98.5%	4.5 个百分点

来源：P5，第 15 页表 4。摘要中数据集列举顺序与提升幅度容易造成对应误读，应按逐数据集表格读取。表中指标保留作者的 mAP 命名；第 10 页配置的 IoU=0.7 不能直接被认作 AP 评估阈值，需进一步核实评估脚本。

最终参数为 2.19 M，GFLOPs 为 6.9，模型大小为 4.7 MB；YOLOv8n 对应 3.01 M、8.1 G、6.3 MB。[P5，第 15 页表 5]

然而作者结论报告：相同 batch=4、640×640 配置下，MBDANet 为约 243 FPS，YOLOv8n 为 514 FPS。作者同时用约 4 ms 描述延迟，但计时的批处理与单图归一化细节仍需明确。因此应保留原文口径，不直接视作 batch=1 的完整链路响应。[P5，第 17 页]

7.7 最值得细读的消融与局限

单用 RFD 达到 96.2%，三个结构模块组合而不蒸馏为 95.6%，组合后加入蒸馏为 96.6%。三个结构模块一起并没有在准确率上超过最强单模块；它换来了 6.9 G 而非 9.6 G 的计算规模。因此更准确的解释是“组合寻找精度与复杂度折中”，不能仅凭模块多就说协同使精度逐项累加。[P5，第 16 页表 6]

另一个例子是蒸馏：在 PCB 上，MBDANet 无蒸馏为 95.6%，L1 为 95.5%，L2 为 95.3%，CWD 为 94.8%，BCKD 才升至 96.6%。这足以说明“用了蒸馏”不是性能保证。[P5，第 13 页表 3]

评估风险： 文字描述为先增强再随机划分。若同一原图及增强版本跨越训练和测试集，会高估独立泛化能力。仅凭文字不能确认已经发生泄漏，但应核查源图分组清单，并做先分源图、后增强训练集的对照。[P5，第 11 页]

三个数据集分别训练评估，说明方法在多种数据上可用，不等同于一个模型从 PCB 零调整迁移到钢材。论文也承认缺少实际产线与边缘验证。参数与 FLOPs 更小却 FPS 更低，更说明部署要看实测。[P5，第 17 – 18 页]

学完应能回答： 为什么单模块比组合更准，并不自动否定组合的价值？为什么这也不能自动证明模块之间存在精度协同？

8. P6：边缘 Wang——学习部署流程，同时审查指标定义

8.1 方法链条与数据范围

论文将图像扩增、YOLOv11 训练、INT8 转换与 RK3568 推理串在一起：[P6，第 6–12 页]

> 训练端：原图 → 裁剪/翻转 → SA-GAN 扩增 → 训练 YOLOv11
> 转换端：加载模型 → 预处理/量化配置 → RKNN 模型构建与导出
> 设备端：驱动与运行库 → 加载 RKNN → 输入处理 → NPU 推理 → 后处理

数据范围必须注意： 文中 NEU-DET 实验选用夹杂、斑块、划痕三类，各 300 张源图，传统增强后为 2450 张，再用 SA-GAN 扩充到 3463 张；不是常见六类完整 NEU-DET 任务。[P6，第 7、12 页]

另在天池铝型材四类数据上验证。只根据“也用了 NEU-DET”不能与 Lu 的六类结果直接比较。

8.2 SA-GAN 有什么用，不能证明什么

生成器与判别器对抗学习样本分布，自注意力帮助联系较远位置与上下文。目标是增加训练图像的多样性，而不是每次检测时都运行 GAN。

合成图像是否有效，应检查缺陷形态是否合理、生成图像的框标注是否准确、是否记忆训练图，以及增强收益是否在独立真实测试集保持。论文对扩展数据重新标注，但这不能代替源图隔离与真实测试验证。[P6，第 7 页]

8.3 RKNN 与 INT8 部署步骤的实际意义

原文配置 mean=[0,0,0]、std=[255,255,255]，意图将像素由 0～255 缩放至 0～1；采用每通道量化、MMSE 和 8 位非对称量化，目标平台 RK3568。[P6，第 8–10 页表 2]

学习时要沿整个数据链检查：训练时 RGB 还是 BGR？输入是怎样缩放和填充的？部署是否重复除以 255？量化校准集是否覆盖低对比度缺陷？输出如何反量化、解码和恢复坐标？这些环节出错，模型即使成功加载也会检测失败。

这些是根据流程提出的复现检查问题，不是本文已经完整回答的实现事实；这里也不把原文旧版本工具用法直接当作当前 SDK 操作手册。

8.4 作者报告的数据

比较	原模型	改进/转换后	差值或变化
三类 NEU-DET，YOLOv11 的 mAP@0.5	81.4%	SA-GAN 后 85.1%	+3.7 个百分点
天池四类，YOLOv11 的 mAP@0.5	83.2%	SA-GAN 后 87.0%	+3.8 个百分点
模型文件	10156 KB	RKNN 后 4194 KB	减少约 58.7%
单图检测时间	52.1 ms	RK3568 为 33.6 ms	减少约 35.5%

来源：P6，第 13 页表 5、第 18 页表 7、第 20 页表 8。表 8 的图像列为 200×200，FLOPs 列在转换前后均写 6.3 G，应保留其原表口径；实际网络输入与计算量统计对应关系仍需核实。

33.6 ms 的倒数约为 29.8 次/秒，只是相同串行单图计时范围下的换算，不能自动当成含相机、传输、预处理与后处理的产线 FPS。作者另描述现场到边缘延迟小于 150 ms，但缺少足够细的延迟分布和分段实测材料。[P6，第 12、20 页]

8.5 本篇需要明确指出的四个问题

第一，AP 公式错误。 第 13 页式（12）将 AP 写为 (PRE+REC)/2。这不是标准目标检测 AP。报告中的 85.1%、87.0% 只能称“作者报告的 mAP@0.5”；需要评估代码或预测结果才能确认是否实际使用标准 AP。公式错不必然等于代码也错，但在核实前不应把结果当作已确认标准口径。

第二，内部数字矛盾。 第 3 页贡献段声称 NEU-DET 为 83.52%→91.23%、量化后大小 18.3 MB；表 5 为 81.4%→85.1%，表 8 为 4194 KB。报告优先使用详细结果表，同时保留矛盾说明，不能择取最大数字作为结论。

第三，扩增与划分顺序需核查。 原文先扩增，再将扩展数据按 8:1:1 划分。若同源图像的裁剪、翻转或生成近邻跨集合，结果可能偏乐观。需确认 GAN 训练是否仅接触训练源图，以及测试是否为独立真实图像。[P6，第 7、12 页]

第四，速度比较改变了多个条件。 云端 RTX 4090 与 RK3568、浮点模型与 INT8、运行后端和可能的计时边界同时变化。52.1→33.6 ms 支持“这组配置的报告时间不同”，不能单独证明“RK3568 通常比 RTX 4090 更快”或“全部提升来自量化”。[P6，第 12、19–20 页]

另一个概念问题是第 10 页把转换后的图示概括为三层结构，并据此说计算减少。编译后的图包装和算子融合，不等于深度网络实际只剩三个普通神经层；表 8 的计算规模也未下降，不能按示意图节点数量推断复杂度。

8.6 阅读判断

这篇适合用来认识“训练、转换、设备运行”的职责分工，也能训练部署证据审查能力。用于精度横向对比时应特别谨慎：三类子集、错误 AP 定义、划分风险和内部数值矛盾共同限制了结论强度。

量化后“准确率基本一致”的论证还较多依赖图示与个例置信度，应补充同一独立测试集上浮点/INT8 的完整 AP、逐类召回和漏检统计。个别框置信度相近，不能替代数据集级精度比较。

学完应能回答： 为什么看懂部署流程和接受性能结论是两件需要分别完成的事？为什么不能拿 85.1% 与 Lu 的 79.2% 判断谁更好？

9. 六篇放在一起，能得出哪些可靠认识

9.1 按技术动作比较，不做跨论文排行榜

技术动作	出现在哪些论文	改变什么	主要代价/风险
更高分辨率检测与特征保留	P1、P5	信息流与检测特征	计算、访存、背景噪声
结构化剪枝	P2、P3、P4	学生结构与张量尺寸	关键通道损失、分支依赖
知识蒸馏	P2、P3、P5	训练监督	教师成本、错误传递、特征失配
调整剪后训练策略	P4	优化过程与计算预算	超参数敏感、训练时间
量化与部署编译	P1、P6	数值精度、执行图	量化误差、算子支持、校准偏差
实际设备/系统评估	P2、P3、P6，各证据不同	工程运行链条	计时边界与真实场景覆盖

P2 有实验平台和界面集成，P3 有 AGX Xavier 速度，P6 报告 RK3568 时间；证据形式不同，不能统称为同等级“已完成产线部署”。P1 的 TensorRT 也是实测，但设备是桌面 GPU。

9.2 三组容易混淆的关系

轻量不等于快。 P5 的参数和 FLOPs 降低，但其报告 GPU FPS 下降。这是直接例子。可能原因包括动态算子、GN、算子碎片、内存访问和实现优化程度；在没有 profiler 前，这些是待检验的解释，不是已经确认的原因。

多数据集不等于跨域泛化。 分别在 A、B 上训练后都获得好结果，说明方法可适配多个任务；在 A 上训练、不接触 B 的训练标签而在 B 测试，才是在评估另一种跨域能力。

完整系统不等于工业可靠性已证实。 界面、相机和正常出框证明集成可用；长期连续运行、跨批次良率、误报成本、极端光照、漏检风险仍需单独证据。

9.3 同一数据集名称为何仍可能不可比

至少核对六项：类别子集、原始样本划分、增强发生时机、输入分辨率、指标计算脚本、推理与训练条件。

本组的典型例子是 NEU-DET：P4 使用六类，P6 使用三类扩展集；P5 的钢表面实验又有自己的训练流程。即便都写 mAP，也不能直接按数值给模型排序。

9.4 从消融实验学习因果判断

假设加入一个模块后提高 0.5 个百分点，至少还要看：有没有多随机种子？训练轮数与数据是否一致？选择的是最好一次还是平均值？是否同时改变了输入或训练策略？是否以额外计算换精度？

逐步加模块只能说明“在已有配置上加入该模块后的变化”。要讨论模块间交互，最好补充单独加入和两两组合，观察增益是否依赖其他模块。P5 比较丰富的组合消融值得学习，但结果同时提醒我们：组合并不保证准确率相加。

以上六篇在本报告所核对的关键结果表中，大多没有提供足够的多次运行均值和方差。因此对 0.1～0.5 个百分点的小差异，先称“报告值差异”，不直接称“稳定显著提升”。

10. 怎样把阅读转成自己的实验能力

10.1 先做一个能解释清楚的基线

选择你有合法可用数据和实现的单一框检测任务。固定原始图像的训练、验证、测试划分，并记录源图或工件身份。同一工件的连拍、裁剪、增强版本应进入同一组；增强只作用于训练数据。

基线先输出完整记录：模型版本、预训练来源、输入大小、类别、训练轮数、随机种子、P/R/AP、逐类结果、参数量、统计工具，以及 batch=1 的延迟。检查几张真值框和预测框，确认坐标变换正确。

10.2 一次只验证一个具体假设

可以学习的假设	最小对照	要观察的结果
更细的特征能改善极小缺陷	基线 vs 增加高分辨率分支	按框面积分组的 AP/Recall、延迟、误报
剪后精度损失可被蒸馏恢复	相同学生结构，无 KD vs 有 KD	多种子 AP、训练成本、逐类变化
通道对齐优于盲目追求最少参数	近似预算、不同通道粒度	目标设备实际延迟与精度
量化是否损害低对比度缺陷	同一权重浮点 vs INT8	同一测试集的 AP 差值和漏检案例

这些是学习实验，不是已确认的新颖论文贡献。要形成研究工作，还需检索现有方法、明确未解决问题、设计公平基线与重复实验。

10.3 一份可复制的实验记录模板

实验编号：
要检验的假设：
对照组与唯一改变：
数据来源、原始样本分组与划分清单：
模型/代码版本、预训练权重：
输入尺寸、batch、epoch、优化器、seed：
压缩单位、剪枝比例定义、保护层：
教师模型、蒸馏位置、对齐方式、损失权重：
设备、后端、精度模式、线程/功耗设置：
AP 口径、逐类 Recall、小目标分组结果：
延迟范围、预热、重复次数、均值与 P95：
错误案例：漏检、误报、重复框、定位偏移：
结果支持了什么，仍不能说明什么：

P95 表示 95% 的测量延迟不超过该值，有助于了解偶发慢帧。完整链路可分为采集、传输、预处理、模型、后处理、结果返回；某段耗时变短不一定让总耗时等比例下降。

10.4 建议的学习顺序与完成标准

1. **先学 Chen。** 手画一层卷积与 BN，解释 gamma、通道删除、下一层输入调整；能分清参数个数和 MB。
2. **再学 Cao。** 用一张图画教师、学生、特征损失与两类输出损失；能独立解释两个基线的百分比。
3. **接着学 Lu。** 画出传统流程和 RDDPA 流程，解释预训练来源与训练预算；能说明“10.8×”的含义。
4. **学习 PCB Wang 与 Shi。** 把每个改动定位到 Backbone/Neck/Head；能说出它针对哪个信息瓶颈，以及增加什么代价。
5. **最后学边缘 Wang。** 从输入像素跟踪到输出框，列出预处理、量化、解码与计时口径；能指出 AP 定义问题。

不要求一开始复现全部模型。先能解释清楚一张表和一个模块，并完成一个受控实验，学习收益通常更明确。

11. 自测题与参考答案

题 1：计算与指标解释

某模型参数从 4.1 M 变为 2.4 M，FLOPs 从 14.7 G 变为 7.2 G；mAP@0.5 从 70.8% 变为 67.5%，蒸馏后回到 70.6%。求压缩比例、剪枝损失和恢复幅度，并解释蒸馏是否减小了结构。

答案： 参数减少 (4.1 2.4)/4.1≈41.5%；FLOPs 减少 (14.7 7.2)/14.7≈51.0%；剪枝损失为 3.3 个百分点，蒸馏恢复 3.1 个百分点，最终仍比完整模型低 0.2 个百分点。这里结构压缩来自剪枝；固定结构上的蒸馏不改变 2.4 M 参数量。

常见错误： 把 3.3 个百分点写成相对下降 3.3%；把减少 51% FLOPs 写成速度提高 51%；忽视分母是完整 SAF 而非原始 YOLOv8n。

题 2：实验设计诊断

研究者把 600 张缺陷原图各增强成 5 张，得到 3000 张，再随机按 8:1:1 划分；测试 AP 很高。能否认为有 300 张独立测试样本？怎样改进？

答案： 不能仅按图像数量认定独立性。同一源图的不同增强版本可能跨集合，使训练与测试共享几何、背景和缺陷位置。应先按原图、工件或采集批次分组划分，再仅增强训练集；测试保留独立真实样本，报告其来源、数量与类别分布。若图像来自连续视频，还需防止相邻帧跨组。

常见错误： 认为增强增加了独立工件数量；只去掉像素完全相同的图片就认定不存在泄漏。

题 3：结构与张量推理

一个 3×3 卷积输入 64 通道、输出 128 通道，忽略偏置；剪去一半输出通道，参数如何变化？下一层原本输入 128 通道，需要改哪里？

答案： 原层参数 3×3×64×128=73728，剪后 36864。下一层权重的输入通道维要从 128 缩为 64，删除对应切片。相关 BN 参数也要同步处理；若输出参与残差相加或 concat，必须追踪两条分支的维度与索引依赖。

常见错误： 只把权重置零却宣称结构已缩小；只改当前层、不处理下游。

题 4：为什么更少 FLOPs 可能更慢

模型 A 为 8.1 G、514 FPS，模型 B 为 6.9 G、243 FPS。是否一定说明测量错误？需要什么证据判断原因？

答案： 不一定。FLOPs 没计入全部访存、调度、动态算子和后端优化差异。先确认同设备、精度、batch、输入、预热和计时范围，再用 profiler 观察算子耗时与内存瓶颈。仅凭这些数值不能确认某个具体算子就是原因。

常见错误： 用理论计算量直接计算实际加速倍数；把 batch 吞吐当单图响应；未分析就归因硬件故障。

题 5：审查一个“边缘更快”的结论

RTX 4090 上浮点模型报告 52.1 ms，RK3568 上 INT8 模型报告 33.6 ms。如何区分硬件、量化与计时的影响？

答案： 建立对照矩阵：在各设备支持的模式下固定模型、输入和精度；在同设备上比较浮点/INT8；尽量明确后端差异；分别计量纯推理与完整链路。使用同一独立测试集比较精度。若某些组合不被支持，应如实报告无法完全解耦，而不是强行归因。

常见错误： 从一组异构配置断言某块板卡普遍优于桌面 GPU；只比较时间而不比较量化后的 AP。

题 6：识别错误 AP 定义

某个固定阈值下 Precision=90%，Recall=80%。能否据此得到 AP=85%？

答案： 不能。85% 是两者算术平均数，不是标准 AP。还需要不同置信度阈值形成的 PR 曲线、匹配规则与具体积分或插值协议。即使这一个点相同，两组检测结果也可能有不同 AP。

12. 证据索引与原文入口

12.1 关键表格回查位置

来源	方法与设置	关键结果	主要核查点
P1	PDF 第 3–9 页	第 13 页表 4；第 14 页表 5	最终 CA 标记、基线 AP、速度差值
P2	第 2–7 页	第 8 页表 2；第 10 页表 3、4	参数/FLOPs 顺序、特征对齐、部署证据
P3	第 9–13、16 页	第 18 页表 3；第 22 页表 5；第 23 页表 6	测试集调参、文件大小单位、微调基线
P4	第 3–5 页	第 6 页表 2；第 7 页表 4；第 9 页表 5	预训练依赖、LS 贡献、两种 GPU
P5	第 4–11 页	第 13 页表 3；第 15 页表 4、5；第 16 页表 6	增强划分顺序、组合折中、FPS 与 FLOPs
P6	第 6–12 页	第 13 页表 5；第 18 页表 7；第 20 页表 8	三类子集、AP 错式、数字矛盾、异构计时

12.2 本地 PDF

- P1：PCB Wang 原文
- P2：Cao 原文
- P3：Chen 原文
- P4：Lu 原文
- P5：Shi 原文
- P6：边缘 Wang 原文

12.3 阅读完成后的检查

合上原文，选一篇用自己的话讲清六件事：任务是什么；基线是什么；每个改动为什么放在那个位置；关键表格支持了什么；结果在哪些条件下成立；下一项最应该补的实验是什么。

如果只能复述“加了某模块，mAP 提升”，说明还停留在摘要层。能解释它改变的张量、训练信号、计算成本以及证据边界，才开始具备独立阅读与设计实验的能力。